

Callbacks (Introduction)

Asynchronous kya hai?

Aisa kaam jo **abhi shuru** hota hai par **baad me khatam** hota hai (jaise `setTimeout`, script load karna). Baaki code us kaam ka intezaar nahi karta.

Problem

```
javascript

loadScript('/my/script.js');
newFunction(); // Error! script abhi load hi nahi hui
```

`loadScript` ke neeche wala code turant chal jaata hai, script load hone ka wait nahi karta.

Solution: callback

Ek function doosre argument me do jo kaam **khatam hone par** chale.

```
javascript

function loadScript(src, callback) {
  let script = document.createElement("script");
  script.src = src;
  script.onload = () => callback(script);
  document.head.append(script);
}

loadScript("/my/script.js", function () {
  newFunction(); // ab chalega
});
```

Isse "callback-based" style kehte hain.

Callback ke andar callback

Ek ke baad ek script load karni ho to callback ke andar agla call:

```
javascript
```

```
loadScript("1.js", function () {
  loadScript("2.js", function () {
    loadScript("3.js", function () {
      // sab load ho gayi
    });
  });
});
```

Thoda kaam ho to theek hai, par zyada ho to mushkil.

Errors handle karna

Script load fail ho sakti hai. Isliye **error-first callback** convention:

javascript

```
script.onload = () => callback(null, script);
script.onerror = () => callback(new Error(`Load error: ${src}`));
```

Rule:

1. Callback ka **pehla argument error** ke liye.
2. Baaki arguments successful result ke liye (`callback(null, result)`).

Use:

javascript

```
loadScript("/my/script.js", function (error, script) {
  if (error) { /* handle */ }
  else { /* success */ }
});
```

Pyramid of Doom (Callback Hell)

Bahut saare nested async kaam:

javascript

```
loadScript("1.js", function (error, script) {  
  if (error) { handleError(error); }  
  else {  
    loadScript("2.js", function (error, script) {  
      if (error) { handleError(error); }  
      else {  
        loadScript("3.js", function (error, script) { /* ... */ });  
      }  
    });  
  }  
});
```

Code **daayin taraf** badhta jaata hai aur padhna/sambhalna mushkil ho jaata hai.

Har step ko alag function (`(step1, step2)...`) bana ke bhi dekha ja sakta hai, par code bikhra hua lagta hai, aur wo functions ek hi baar kaam aate hain.

Yaad rakho

- Callback = "kaam khatam ho to ye function chala dena".
- Error-first callback ek common convention hai.
- Zyada nesting = callback hell. Iska behtar hal **Promises** hain (agli file).